Findings — one evening of real play against MAG

2026-08-01. Will played MAG in the actual Pokémon Showdown client for the first time (`engine/showdown_bot.js`, local server, `[Gen 9] Champions VGC 2026 Reg M-B`). He won every game.

Every number in this document was produced by running something on 2026-08-01. Nothing is quoted from another document.

---

1. THE HEADLINE, AND IT IS NOT "THE MODEL IS BAD"

Nine defects surfaced in about an hour. **Six of them were wiring**, and they meant the policy was never actually making the decisions:

> *Will: "bro so our bot sucked forever" — and then, later, "bro this thing sucks".*

The correct reading is narrower and worse. The bot did not suck *forever*: it sucked **from the hour it was written**, which was the same evening. Nothing in the trained model, the corpus, or the self-play results is implicated by items 1–6 below.

A correction to what was said in the session. During the session it was claimed that MAG had never mega evolved across 586,816 self-play games. **That is wrong.** `mew.js:226` reads `const MEGA_P = parseFloat(arg('mega', '1'))` and passes it to the constructor, so self-play megas normally; `tests/test-wiring.js` measures megas on 85% of sides. The mega defect was confined to the Showdown bot. The claim was made from reading a default and not from running anything, which is the exact failure this project's own rules exist to prevent.

---

2. FIXED

#	What Will saw	Root cause	Fix
1	*"no mega evolution tho"*	`magnemite.js` defaulted `megaP` to **0** while the comment four lines above said *"Defaulted to 1"*. The bot passed no `mega` option, so it took that default forever.	default changed to 1
2	*"it eq its own blastoise"*	The player was **sampling** from the softmax, not taking its best move. `greedy` never arrived — see #3.	see #3
3	—	`makeScoringPlayer(opts = {})` **never used** `opts`**.** The class reads its own *second constructor argument*. `makeScoringPlayer({greedy:true, switching:true})` then `new Player(stream)` silently configured **nothing**.	factory options now merge into constructor options; explicit constructor options still win
4	*"dont u need my open team sheet"*	The server offers open sheets as a `\`	uhtml\	otsrequest\	` button. Nothing in the bot answered,	bot answers `/acceptopenteamsheets` once per room

#	What Will saw	Root cause	Fix			
					so MAG played blind to the opponent's sets — and locked its bring at team preview regardless.	
5	*"two light screens on the same turn? bro"*	`magnemite.js` defaults `this.joint = false`. Each slot was scored on its own 56 features with **no knowledge of the other slot**.	bot passes `joint: true` (74-weight vector loads clean)			
6	*"grimmsnarl doesnt have an item???"*	The Item Clause retry in `champions_sim.js` called `SP.fillSet(name, known, seed+k*7919)` **with the colliding item still in** `known`. All six redraws returned the same item; `alt` stayed `''`. The "rare" fallback to no-item fired *every* time.	resample against a copy of `known` with `item` deleted			
7	(unseen)	27.5% of generated teams fail validation (11 of 40 measured), and the bot **rejected the challenge** rather than redrawing.	bounded redraw, 12 attempts, then a loud throw			

Measured after fix #6: **2 of 174 Pokémon itemless (1.1%)** across 40 generated teams.

2.0 An unmeasured change shipped against a measured verdict — `switching`

`showdown_bot.js` asked for `switching: true` in its first version. `mew.js:135` records the verdict it was contradicting:

> *"--switching let MAG choose to switch. **Measured as a 10-point LOSS** against a random opponent, so it is off until the switch policy is worth more than not switching."*

mew requires an explicit `--switching` flag and defaults it off. The bot asked for it unconditionally. Because of defect #3 the option never arrived, so its first live outing was the evening of 2026-08-01, immediately after the merge fix.

Two games, one each way: MAG **won** the game whose only switch was a forced post-KO replacement (1 switch-bearing choice) and **lost** the game in which it chose to switch four times. **That is n=1 per arm and proves nothing.** It did not need to — the lever was already measured at −10 points and nothing has re-measured it since. Reverted to off.

Post-KO replacement is a different lever (`forcedSwitch`, also off by default) and is untouched: when a Pokémon faints, passing is not a legal choice, so the −10 verdict does not apply to it.

The general failure: a new caller re-enabled, by default and without measurement, a lever the project had already measured as harmful and deliberately defaulted off elsewhere. The same shape as #1 (a default contradicting its own documentation) and #5 in §5 (a capability forced on in tests and off in production) — knowledge that exists in the repository, not applied at the new call site.

2.0b Enabling the joint layer silently disabled mega evolution — a regression from the fix above

> *Will, later the same night: "now it never emgaed"*

`_withMega` had **exactly one call site**, on the independent decision path. `_decidePair` — the joint path — ended with `return candsA[pa].choice`, bypassing it, and the partner's parked half returned bare at the `_jointPick` branch too. So the two levers could not both be on: bot7, with joint off, mega evolved a Gyarados; every build after `joint: true` was added never mega evolved again.

Ruled out first: **94% of generated teams carry a mega stone** (measured over 48 valid teams), so the team builder was not the explanation.

Both joint return paths now go through `_withMega`. Only one slot can hold a stone — the bot's team builder permits one, and Item Clause bars a duplicate — so the two calls cannot both claim the battle's single mega.

This is the fourth mega defect in this project. The first three are recorded at `mew.js:446` (never passed the option, 199,524 games), `magnemite.js:226` (base class could only mega from the left slot, 56% of sides), and §2 #1 above (the default contradicting its own comment).

Why the wiring test missed it, again. `tests/test-wiring.js:135` asserted the mega *rate* on the default run. Line 138 ran the joint configuration and asserted only that the joint layer had run. **The broken cell was the intersection, and no assertion stood in it.** The joint path is a second code path that returns through different code; it inherits no claim from the first. Fixed by re-checking mega, aiming and open team sheets under `--joint`.

2.1 The one that is not confined to the bot

`allyHit` **could not see type immunity.** `board.js:2786` used `dex.getEffectiveness(mType, aTypes) >= 0`. Measured against the Champions dex:

move type	ally type	`getEffectiveness`	can actually be hit
Ground	Flying	0	**no — immune**
Ground	Water	0	yes
Electric	Ground	0	**no — immune**
Ground	Grass	−1	yes (resists)

Identical scores, opposite truths — immunity lives in `getImmunity`, not `getEffectiveness`. So a Flying partner standing beside Earthquake read as **hit**, and since `spreadFreeBesideAlly` requires `allyHit === 0`, the flagship case named in its own comment (`board.js:489`, *"EARTHQUAKE BESIDE A FLYING PARTNER"*) **could never fire once**. This answers Will's earlier question about why that feature never moved.

Note the asymmetry it created: the *ability* route to the same immunity (Levitate, via `abilityBlockProb`) worked correctly. Type immunity and ability immunity are the same fact about the board and only one was being read.

Introduced 2026-07-26. `engine/board.js` is required by **40 files**; the fitted vector is read by **24**. Fixed, but see §4 — the fix is inert until a refit.

---

3. NOT FIXED — open, with the specific fix

3.1 MAG cannot see that it is Choice-locked *(model gap)*

> *Will: "it did swap out after i encored it, but not aftre i tricked it"*

Of the 56 features, the only ones matching `/encore|choice|lock|taunt|disable|torment|trick|item/` are `abilityBlock` and `stallIntoEncore` — and `stallIntoEncore` is about **inflicting** Encore, not suffering it. There is **no representation of holding an unwanted Choice item**.

Hypothesis, untested: the Encore switch happened because Showdown collapses the legal move list to one, changing the choice space rather than the score. A probe that logs the scored candidates on a choice-locked turn would settle it.

Fix: a feature that fires when this Pokémon's item is a Choice item it did not start with, or more generally when its legal move count has collapsed below its known move count. Requires a refit.

3.2 No pair feature for "both slots set the same side condition" *(model gap)*

The 18 joint features are: `bothSameTarget`, `overkill`, `focusFireKills`, `partnerCoversMe`, `redirectThenAttack`, `bothStatus`, `bothSwitch`, `boostsPartnerDamage`, `boostMayConvertKill`, `speedSetupHelpsPartner`, `weatherSetupHelpsPartner`, `healsPartner`, `redirectThenSetup`, `doubleKO`, `flinchThenSetup`, `terrainSetupHelpsPartner`, `screenWhileThreatened`, `spreadFreeBesideAlly`.

None of them is *"both slots set the same side condition this turn."* `deadSide` — **the most negative weight in the entire vector at −2.879** — cannot help, because it asks whether the side condition is *already up*, and on the turn both slots choose it, it is not up for either.

Enabling `joint` was **necessary and may not be sufficient** for the double-Light-Screen case.

Fix: a pair feature `bothSameSideCondition`, derived from `m.sideCondition` matching across the two candidate moves. Purely a dex-field comparison, no move named. Requires a refit.

3.3 No floor on how rare a sampled set may be *(team generation)*

> *Will: "why smooth rock bro what sort of set is this"*

The set was **real**. Of 162 Gyarados sheets: `n=1 (0.62%) SmoothRock / Intimidate / Careful / Sandstorm, Endeavor, Taunt, Waterfall`. One human, one time. The actual Gyarados is `19.8% Gyaradosite / Adamant / Waterfall, Lash Out, Dragon Dance, Protect`.

`species_sets.sample()` is correctly **joint** — it draws a whole observed set as a unit, not parts glued together — but it samples the entire tail proportionally, and a set seen once is indistinguishable from a typo, a troll, or a misclick.

Across the corpus: **10,504 distinct sets, of which 3,949 (37.6%) were seen exactly once.**

`species_sets.cover(species, frac)` already exists for exactly this. Measured:

species	distinct sets	sets covering 80% of real usage
Gyarados	38	12
Incineroar	329	48
Grimmsnarl	179	27
Glimmora	57	5
Politoed	99	37

Fix: the *bot's* team builder should sample within `cover(sp, 0.80)` rather than the full tail. **Deliberately not applied globally** — DITTO's gauntlet must keep the whole tail, because optimising a team against opponents who all run the modal set is optimising against a metagame that does not exist. This is a caller's decision, and the two callers want opposite things.

---

4. THE THING THAT BLOCKS THE OTHERS: a refit is now required

`board.js:2786` changed **feature semantics without changing the feature list.** Every shipped weight was fitted against the old, wrong value of `allyHit`.

`magnemite.js:297` guards feature-list drift by comparing joined names, and `magnemite.js:299` guards vector length. **Neither can detect a semantics change under an unchanged name.** Nothing in the project currently fails when a feature quietly starts meaning something else.

That is a gap of the same shape as the ones this project keeps finding, and it deserves a guard: a stored hash of each feature's values over a fixed fixture board, checked at load, so a changed meaning fails loudly instead of silently invalidating 24 files' worth of weights.

Until the refit, `allyHit`'s current weight of **−0.0187** should be read as a number produced by a contaminated feature. For scale, real penalties in the same vector are `deadSide` −2.879 and `abilityBlock` −2.184.

Hypothesis, untested: the immunity bug is **why** `allyHit` came out near zero — the feature mixed free hits (Flying partner, immune) with costly ones (Will's Blastoise) under one name, and averaging two opposite cases drags a coefficient toward nothing. The refit is the test.

---

5. WHY THE TEST SUITE DID NOT CATCH #1

`tests/test-wiring.js` exists **precisely** for this bug. Its own header documents the prior incident:

> *"We never passed the option, so across 199,524 self-play games the bots mega-evolved essentially zero times — while 93% of real ladder games contain one."*

That was fixed **in mew**, by passing the option. The broken default was left in place. The test then verified megas work — but it runs through mew, at line 107, under the banner `default configuration (open sheets forced, mega on)`.

A test that configures a capability on and then checks the capability cannot ever catch a caller who forgets to configure it. The next new caller walked into the identical hole. This is the third occurrence of the same mega defect.

The fix applied is structural rather than local: the factory's options now merge, so an option cannot be dropped for choosing a valid-looking spelling. The deeper rule is that **defaults should be the working value**, so that a caller who configures nothing gets a working bot.

---

6. WHAT I COULD NOT VERIFY

- **Whether the Encore switch was understanding or move-list collapse** (§3.1). Stated as a hypothesis. Needs a probe on a choice-locked turn.
- **Whether the immunity bug caused `allyHit`'s near-zero weight** (§4). Stated as a hypothesis. Needs the refit.
- **Whether `joint: true` is a net improvement for a greedy player.** The joint fit exists and loads, but **no H2H has compared joint-on against joint-off in the greedy configuration.** It was enabled

because it is the only mechanism that can express slot coordination at all, not because it is measured better here. This is an unmeasured change and should be treated as one.

- **Whether MAG is actually weak.** Will won three games, but games 1–3 were played by a bot that was variously sampling instead of choosing, uncoordinated, blind to open sheets, and unable to mega. **No game to date has tested the policy.** Nothing about MAG's playing strength was established this evening, in either direction.